

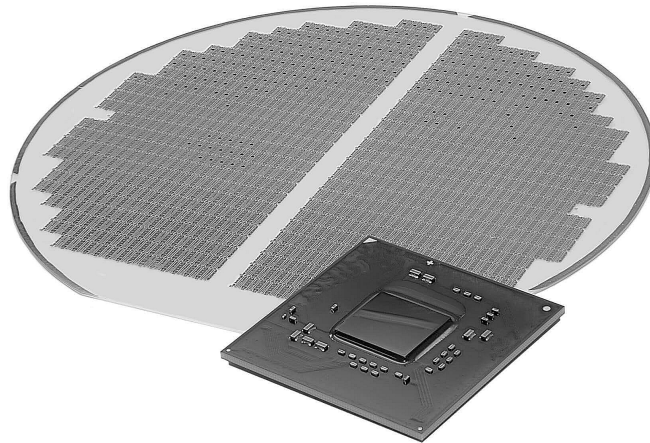


UTN FRC

INGENIERÍA ELECTRÓNICA

DISPOSITIVOS ELECTRÓNICOS

TRABAJO PRÁCTICO DE LABORATORIO 6
Flujo de Diseño Analógico con CMOS



Índice

1. Introducción.	3
1.1. MOSFET	3
1.2. El inversor CMOS	4
1.3. PDK (Process Design Kit)	5
1.3.1. PDK SKY130	6
1.4. Flujo de Diseño.	8
2. Consigna del TP: Especificaciones de Diseño.	11
3. Actividad 1: Instalación de las herramientas.	11
3.1. Pasos para instalar las herramientas necesarias.	12
4. Actividad 2: Flujo de Diseño Analógico.	12
4.1. Schematic Design	12
4.2. Simulation	14
4.3. Layout: Placement / Routing	16
4.4. DRC	17
4.5. LVS	18
4.6. Post-layout	18
4.7. Generación del archivo .gds para fabricación	19
5. Conclusiones.	21
6. Bibliografía / Referencias.	22
7. Anexos	23
7.1. Resumen de comandos básicos de Magic	23
7.2. Resumen atajos usados en Xschem	24
7.3. Comandos para simulaciones en ngspice	24

Objetivos.

- Comprender y aplicar el flujo de diseño analógico para circuitos integrados en tecnología CMOS.
- Conocer y utilizar las herramientas opensource de diseño de CI para validar esquemáticos y layouts.
- Familiarizarse con las reglas de diseño físico (DRC), simulaciones y verificaciones para garantizar la compatibilidad con procesos de fabricación.
- Integrar el conocimiento teórico con la práctica utilizando un kit de diseño (PDK) SKY130 y herramientas CAD para un diseño confiable y escalable.

Funciones.

Se debe asignar roles dentro del grupo y estos deben ir rotando entre los diferentes miembros del equipo en forma consecutiva en los diferentes Trabajos Prácticos de la Cátedra.

- **Coordinador/a:** Será el encargado de organizar las tareas requeridas por cada TP. Además, debe ser quién lleve adelante la exposición y defensa del trabajo realizado en el coloquio oral frente al docente.
- **Operadores/as:** Son las personas que llevarán adelante las actividades dirigidas por el coordinador. Además, estas personas son quienes proporcionarán información a quién esté asignado como responsable del registro de la actividad en el laboratorio.
- **Documentación:** Esta función estará compuesta por uno o más estudiantes. Son los responsables de realizar la documentación final y formalizar las notas en laboratorio. Es recomendable que implemente una bitácora de las diferentes actividades/mediciones que se realizan en el laboratorio, principalmente es recomendable que se tome registro fotográfico de cada paso para su posterior presentación en el informe del TP.

Rúbricas

La calificación será grupal y se aplicarán los criterios planteados en la siguiente tabla. Se especifica el porcentaje que comprende cada uno de los criterios de evaluación.

Tarea	Puntuación	Máximo
Presentación del informe con archivos correspondientes		20 %
Comprender la estructura interna y funcionamiento del inversor CMOS		30 %
Comprender las etapas involucradas en el flujo de diseño analógico		30 %
Defensa de las conclusiones		20 %
TOTAL		100 %

1. Introducción.

En este trabajo práctico se diseñará un inversor analógico utilizando tecnología CMOS. El inversor CMOS es una compuerta lógica fundamental que utiliza un transistor PMOS y un transistor NMOS conectados en serie, donde su operación complementaria permite obtener una alta eficiencia energética y velocidad en el cambio de niveles lógicos. Se utilizará un modelo de flujo de diseño, desde las especificaciones y simulación hasta la verificación del layout con reglas de diseño reales del proceso SKY130.

1.1. MOSFET

El *MOSFET* (Metal Oxide Semiconductor Field Effect Transistor) o Transistor de Efecto de Campo Metal-Óxido-Semiconductor es el dispositivo más utilizado en la industria microelectrónica. La figura 1 muestra una estructura simplificada de un dispositivo MOS de tipo n (*NMOS*) fabricado sobre un sustrato de *tipo p* (también denominado *bulk* o *cuerpo*), el dispositivo consta de dos regiones n fuertemente dopadas que forman los terminales de fuente y drenador, una pieza fuertemente dopada (conductora) de *polisilicio* (denominada simplemente *poli*) que funciona como compuerta, y una fina capa de dióxido de silicio (SiO_2) (denominada simplemente *óxido*) que aísla la compuerta del sustrato. La acción útil del dispositivo se produce en la región del sustrato bajo el óxido de la compuerta. Obsérvese que la estructura es simétrica con respecto a *S* y *D*. En la derecha de la figura 1 se observan los símbolos eléctricos usados tanto para un *NMOS* como para un *PMOS*.

La dimensión lateral de la compuerta a lo largo de la ruta fuente-drenador se denomina longitud *L*, y la perpendicular a la longitud se denomina ancho *W*. Dado que las uniones S/D se *difunden lateralmente* durante la fabricación, la distancia real entre la fuente y el drenaje es ligeramente inferior a *L*. Se puede decir que : $L_{eff} = L_{drawn} - 2L_D$, donde L_{eff} es la *longitud efectiva*, L_{drawn} es la *longitud total*, y L_D es la *cantidad de difusión lateral*. La longitud efectiva, L_{eff} y el espesor del óxido de la puerta, t_{ox} , desempeñan un papel importante en el rendimiento de los circuitos MOS. En consecuencia, el objetivo principal del desarrollo de la tecnología MOS es reducir ambas dimensiones de una generación a otra sin degradar otros parámetros del dispositivo. Los valores típicos en la actualidad son $L_{eff} \approx 5nm$ y $t_{ox} < 2nm$. En la práctica generalmente se usa para denotar la longitud efectiva por *L* en lugar de L_{eff} .

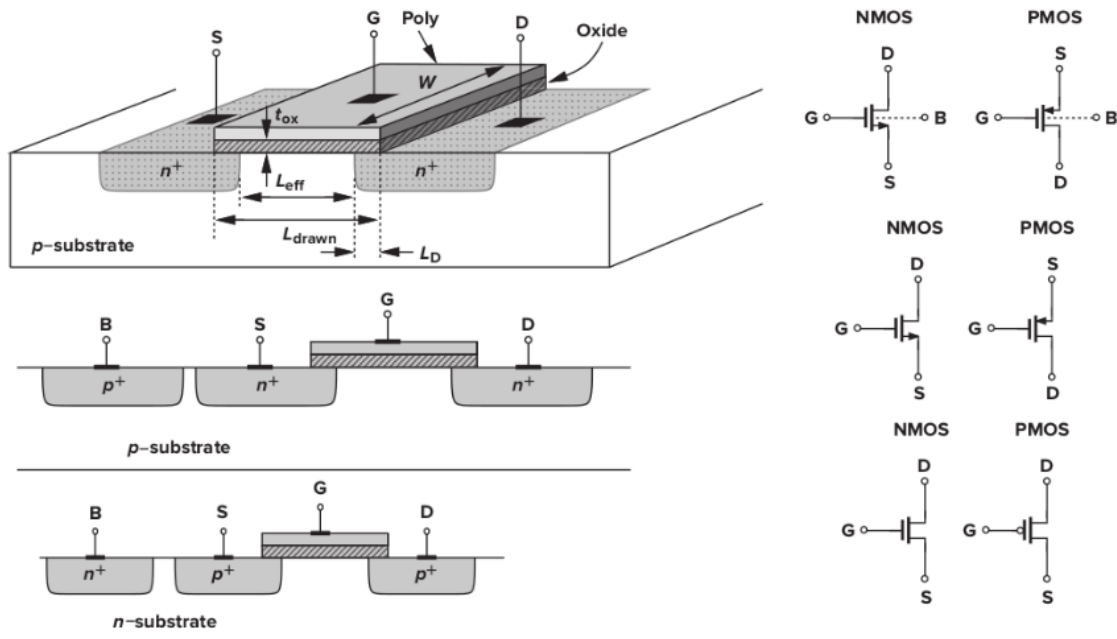


Figura 1: Estructura de un NMOS y PMOS, junto con sus símbolos eléctricos.

El diseño de un MOSFET viene determinado tanto por las propiedades eléctricas que se requieren del dispositivo en el circuito como por las *reglas de diseño* impuestas por la tecnología (en este caso se usará la tecnología de SKY130). Por ejemplo, W/L se elige para establecer la transconductancia u otros parámetros del circuito, mientras que el L mínimo viene dictado por el proceso. Además de la compuerta, también deben definirse adecuadamente las áreas de fuente y drenaje. En la figura 2 se muestran la *vista lateral* y la *vista superior* de un MOSFET. El polisilicio de la compuerta y los terminales de fuente y drenador deben estar unidos a cables metálicos (de aluminio) que sirven como interconexiones con baja resistencia y capacitancia. Para ello, se deben abrir una o más *ventanas de contacto* en cada región, rellenarlas con metal y conectarlas a los cables metálicos superiores. Obsérvese que el polisilicio de la puerta se extiende más allá del área del canal en cierta medida para garantizar una definición fiable del *borde* del transistor. Las uniones de la fuente y el drenaje desempeñan un papel importante en el rendimiento. Para minimizar la capacitancia de S y D, se debe minimizar el área total de cada unión.

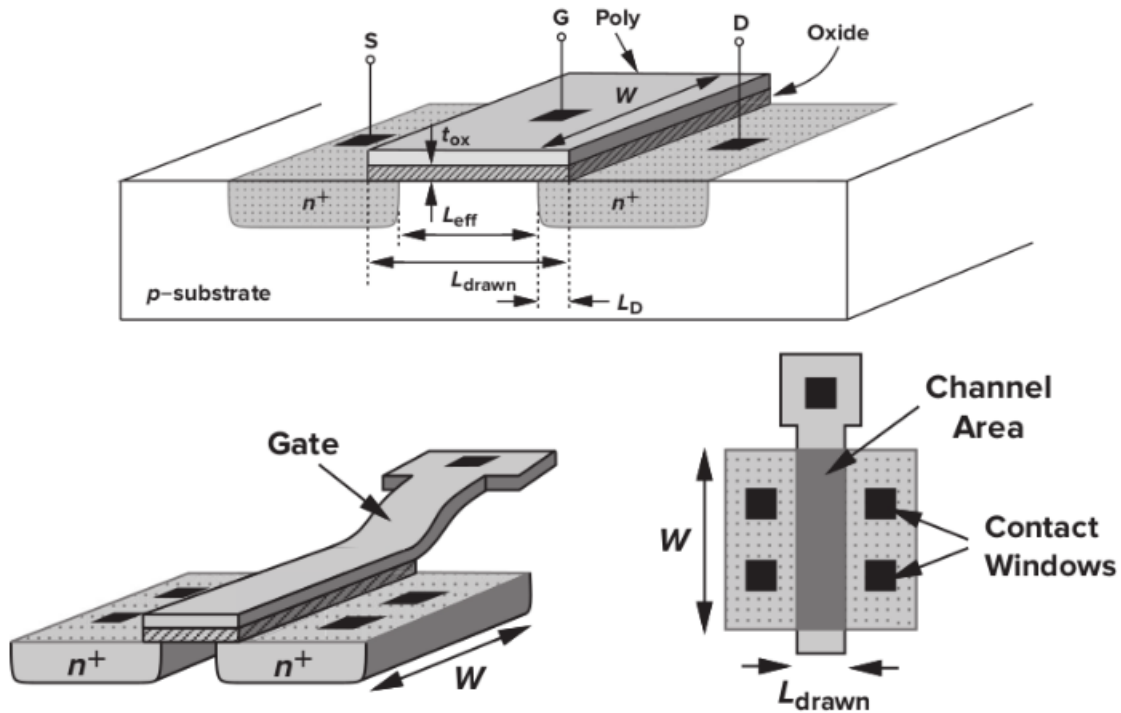


Figura 2: Vistas superior y lateral de un dispositivo MOS.

1.2. El inversor CMOS

En las tecnologías MOS complementarias (*CMOS*), se dispone tanto de transistores *NMOS* como *PMOS*. Desde un punto de vista simplista, el dispositivo *PMOS* se obtiene negando todos los tipos de dopaje (incluido el sustrato), pero en la práctica, los dispositivos NMOS y PMOS deben fabricarse en la misma oblea, es decir, en el mismo sustrato. Por esta razón, un tipo de dispositivo puede colocarse en un *sustrato local*, normalmente denominado *well* o *pozo*. En los procesos *CMOS* actuales, el dispositivo *PMOS* se fabrica en un pozo n (*n-well*), ver figura 3. Tenga en cuenta que el (*n-well*) debe conectarse a un potencial tal que los diodos de unión S/D del transistor *PMOS* permanezcan polarizados en inverso en todas las condiciones. En la mayoría de los circuitos, el pozo n está conectado a la tensión de alimentación más positiva.

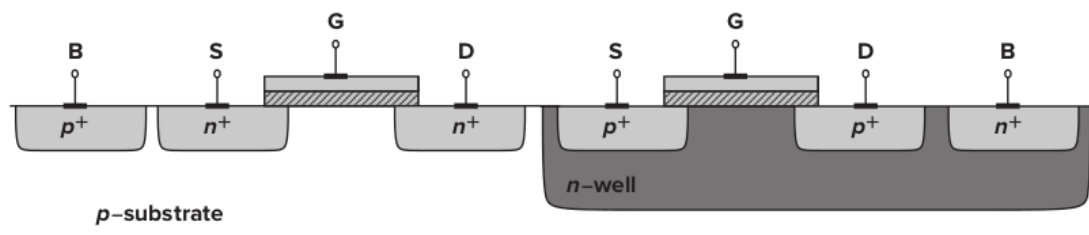


Figura 3: Estructura de un CMOS.

El *inversor CMOS* es uno de los bloques básicos y fundamentales en el diseño de circuitos digitales y analógicos con tecnología *CMOS*. Está compuesto por dos transistores complementarios: un transistor de tipo *NMOS* y otro de tipo *PMOS* conectados en configuración complementaria, tal como se muestra en la figura 4.

Cuando la entrada está en un nivel lógico alto, el transistor *NMOS* se encuentra en conducción mientras que el transistor *PMOS* está apagado, lo que deja la salida conectada a tierra (nivel bajo). Por el contrario, cuando la entrada está en nivel bajo, el transistor *PMOS* conduce y el *NMOS* se apaga, conectando la salida al voltaje de alimentación (nivel alto). Esta operación complementaria permite consumir muy poca corriente estática, ya que en estado estable no hay camino directo entre la alimentación y tierra.

Además, el inversor *CMOS* es utilizado no solo como elemento lógico, sino también como una célula fundamental para el diseño de circuitos analógicos, gracias a su rápido tiempo de conmutación y bajo consumo de potencia.

La relación de tamaños entre los transistores *PMOS* y *NMOS* es crucial para asegurar una correcta inversión de la señal y tiempos de subida y bajada balanceados. Generalmente, el transistor *PMOS* debe ser más ancho que el *NMOS* debido a que los portadores de carga en *PMOS* tienen menor movilidad.

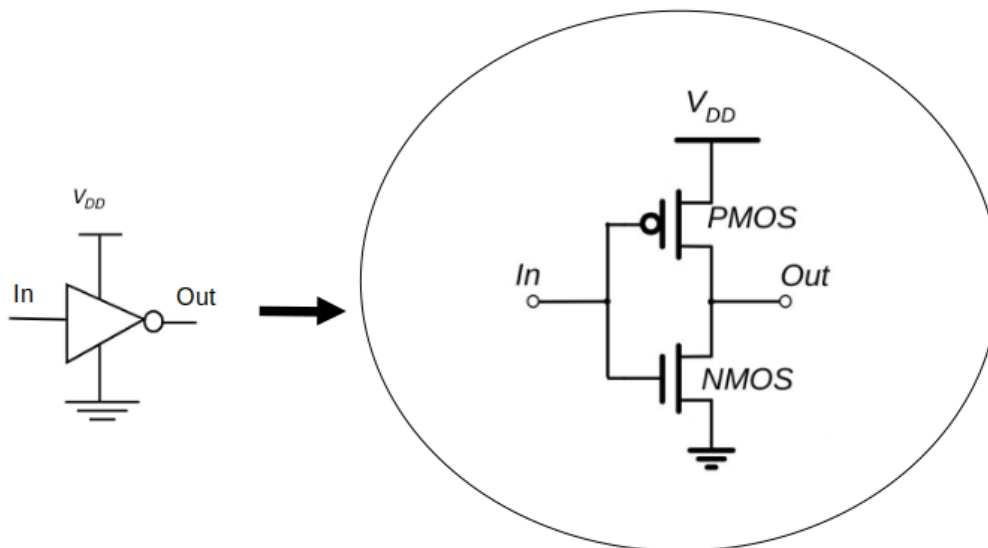


Figura 4: inversor con CMOS.

1.3. PDK (Process Design Kit)

El PDK es el conjunto esencial de archivos y reglas que proporciona la empresa fabricante de semiconductores para que los diseñadores puedan crear circuitos integrados compatibles con un proceso de fabricación específico. En detalle, el **PDK** actúa como un manual o guía técnica que contiene:

- **Modelos de dispositivos** (transistores, resistencias, condensadores, etc.)
- **Parámetros eléctricos y físicos de los dispositivos.**
- **Bibliotecas de celdas estándar y símbolos.**
- **Reglas de diseño físico (DRC)** para asegurar que el layout cumpla con las restricciones de fabricación.
- **Archivos para simulación y verificación** (modelos SPICE, reglas eléctricas).
- Información sobre capas, nombres y atributos para el diseño del layout.
- Archivos de verificación y formatos compatibles con herramientas CAD usadas en diseño de circuitos integrados.

1.3.1. PDK SKY130

El *PDK SKY130* es un Kit de Diseño de Procesos de código abierto desarrollado en colaboración entre *Google* y *SkyWater Technology Foundry* (ver <https://skywater-pdk.readthedocs.io/en/main/>), que proporciona todas las reglas, modelos y bibliotecas necesarias para diseñar circuitos integrados en tecnología CMOS de 130 nanómetros, ver figura 5.

Este PDK permite a diseñadores y desarrolladores crear, simular y verificar diseños físicos compatibles con un proceso de fabricación real, con características y reglas específicas definidas para esta tecnología. SKY130 incluye modelos SPICE para simulaciones eléctricas, reglas de diseño físico (DRC), información sobre capas y metales, así como bibliotecas de celdas estándar.

Dentro de SKY130, las capas metálicas tienen grosores progresivos para optimizar la fabricación, y se soportan dispositivos MOSFETs optimizados para este nodo de proceso. Además, SKY130 es muy popular en la comunidad open source porque facilita el acceso a la fabricación y diseño de circuitos integrados sin costo, reduciendo barreras para startups, instituciones educativas y proyectos experimentales.

La apertura del PDK SKY130 ha permitido la integración con múltiples herramientas CAD opensource para un flujo de diseño completo incluyendo simulación, diseño físico, verificación y síntesis.

Este PDK representa un buen punto de partida a quienes quieran introducirse en el mundo del diseño de circuitos integrados, apoyando la innovación y la educación en diseño de semiconductores.



FOSS 130nm Production PDK
github.com/google/skywater-pdk

Current Status - Experimental Preview

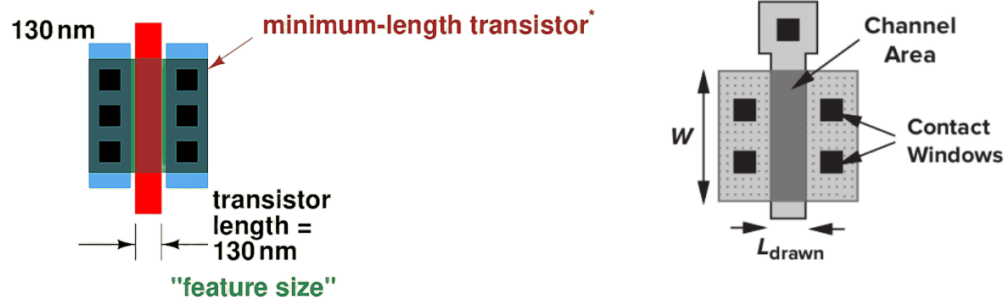
SKY130

Figura 5: tecnología sky130.

El PDK SKY130 cuenta con 5 capas metálicas (Metal 1 a Metal 5), ver figura 6, donde cada capa presenta un espesor progresivo que va desde $0.36\ \mu\text{m}$ en Metal 1 hasta $1.26\ \mu\text{m}$ en Metal 5, optimizando la conductividad y la integridad del diseño físico a medida que se sube en las capas de metal.

Las reglas de diseño físico (DRC) del PDK SKY130 son un conjunto detallado de restricciones que deben cumplirse para garantizar la fabricabilidad de los circuitos integrados diseñados en esta tecnología de 130 nm. Algunas de las reglas principales incluyen:

- Ancho mínimo de línea.
- Espaciado mínimo entre líneas.
- Vías y contactos.
- Superposición.
- Rejillas de colocación.
- Densidad mínima y máxima de metal.
- Restricciones específicas para transistores.

Estas reglas se presentan como archivos en formato texto o formato específico para herramientas de verificación (por ejemplo Magic), y se usan automáticamente para verificar el diseño físico durante el flujo de trabajo. El cumplimiento estricto del DRC previene errores críticos que puedan causar fallos en la fabricación o mal funcionamiento del chip.

La documentación oficial y detallada para las reglas de diseño físico (DRC) del PDK SKY130 está disponible en el sitio de documentación del proyecto SkyWater PDK, donde se incluyen las reglas específicas de diseño, detalles de dispositivos, capas metálicas y otras especificaciones técnicas <https://skywater-pdk.readthedocs.io/en/main/rules/device-details.html>

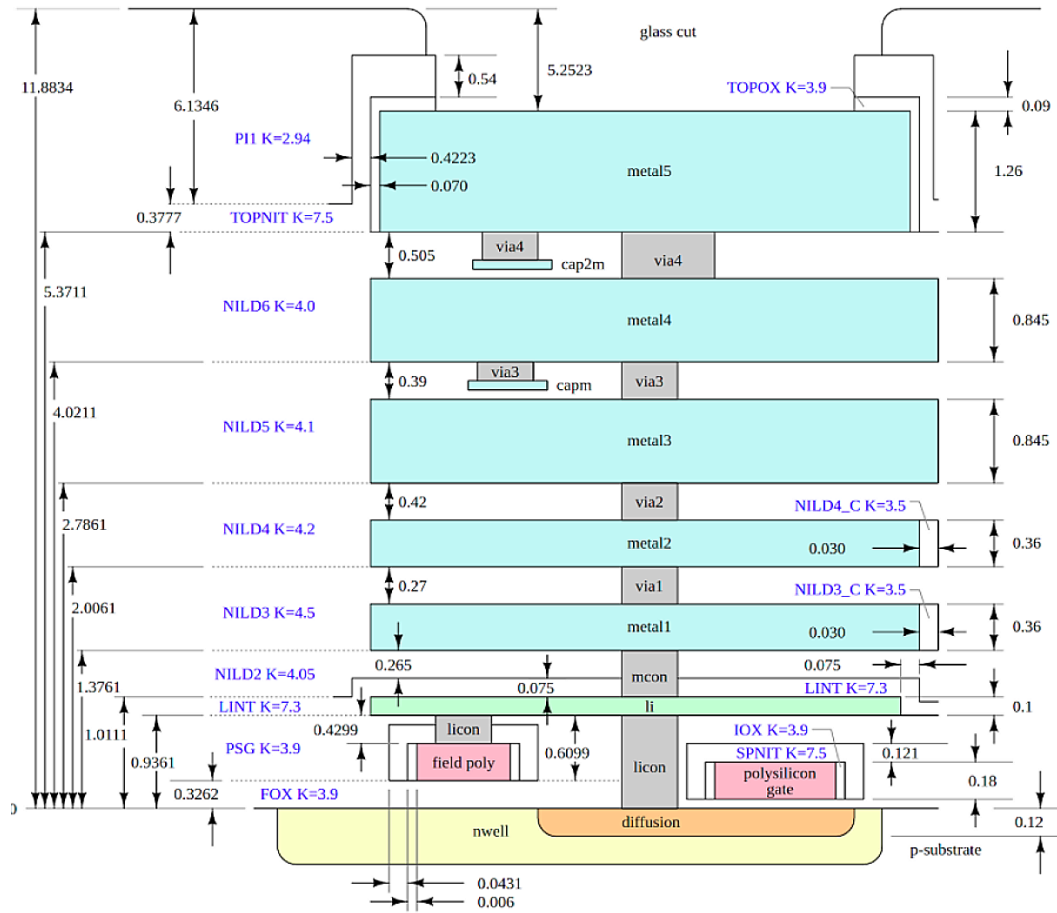


Figura 6: capas metálicas disponibles en sky130.

1.4. Flujo de Diseño.

Para abordar el TP, se va a usar como referencia el flujo de trabajo de la figura 7, en el cual se van a describir por etapas los detalles implicados en cada instancia del proceso:

- Se empieza por las especificaciones técnicas que debe cumplir el diseño. Los requerimientos que normalmente se fijan como parámetros a cumplir en el desarrollo de circuitos integrados son: el área (que impactará en el precio), consumo (o power), velocidad (speed o también llamado performance). En esta etapa se usará la herramienta **xschem** para esquematizar el circuito propuesto.
- Una vez realizado el esquemático se tiene que simular para verificar comportamiento, en este paso se usará la herramienta **ngspice**. Si este circuito cumple con las especificaciones ya se puede avanzar con el layout.
- Para realizar el Layout del circuito se deberá hacer la ubicación física (placement) de los transistores o bloques funcionales del circuito dentro del área del chip. El objetivo será posicionar estos elementos de manera óptima para minimizar el área, reducir el retardo de señal y facilitar el enrutamiento posterior. En cuanto al enrutamiento (routing), se puede decir que consistirá en realizar las conexiones físicas reales entre los transistores colocados. Aquí se definen las pistas o canales por donde pasan las señales eléctricas que enlazan estos transistores para que trabajen en conjunto. La herramienta a usar en esta instancia es **Magic**
- Una vez realizado el layout del circuito, se tiene que hacer el *DRC* (*Design Rule Check* o *Comprobación de Reglas de Diseño*) para verificar que el layout (diseño físico) del circuito

cumple con las reglas y restricciones impuestas por la tecnología de fabricación (SKY130). Estas reglas incluyen distancias mínimas entre elementos, tamaños mínimos de componentes, espaciamientos, anchos de pistas y otras especificaciones físicas esenciales para asegurar que el circuito pueda ser fabricado de manera fiable y funcione correctamente. Se usará también **Magic**.

- Una vez que se el DRC esté ok, se procede a realizar el *LVS (Layout vs Schematic)* para verificar que el diseño físico del circuito integrado, representado en el layout, corresponda exactamente al esquema eléctrico original. Esto implica comparar la netlist extraída del layout con la netlist generada por el esquema para asegurar que todas las conexiones, componentes y geometrías se correspondan correctamente. En esta etapa se usará la herramienta **netgen**.
- Si bien, hasta el momento se han realizado simulaciones previas validando el comportamiento del circuito diseñado, es necesario tener en cuenta factores que pueden influir de manera directa en el desempeño y que no se han modelizado antes. Lo cual implica contemplar una etapa de *Post-layout* para identificar y modelar los efectos parasitarios (inductancias, capacitancias y resistencias no deseadas) presentes en las interconexiones y dispositivos de un circuito integrado tras el diseño físico se usará la *extracción de parásitos*. En este caso se tiene que volver a simular considerando estos efectos. Si durante estas simulaciones postlayout se detectan problemas significativos que afecten el rendimiento o la integridad eléctrica, puede ser necesario regresar a etapas anteriores para ajustar el *placement y routing*, haciendo iteraciones parciales para mejorar el diseño. Si se cumple todo ya está en condiciones de enviar a fabricar el circuito.
- *El tapeout* es la etapa final del proceso de diseño de circuitos integrados (CI), donde se completa y se envía el diseño final del chip (**.gds**) a la foundrie o planta de fabricación para su producción. El archivo (.gds) GDSII (Graphic Database System II) se refiere al formato de archivo estándar de la industria para el diseño físico de circuitos integrados. Este archivo contiene la representación geométrica y jerárquica del layout del circuito integrado, y se utiliza para intercambiar datos con la foundrie para la fabricación del chip. Se adjunta link donde explica mas en detalle de como es el proceso de fabricación dentro de la foundrie <https://www.youtube.com/watch?v=9SPMP5XolHA&t=295s>.
- Una vez fabricados los chips en la oblea, se realiza un control de calidad para seleccionar los chips funcionales, marcando o descartando los defectuosos. Luego la oblea se corta para separar cada chip individual. Los chips funcionales se encapsulan en carcasas protectoras y sus patas o conexiones se unen eléctricamente. Finalmente, se procede a la etapa de *testing* o pruebas funcionales, donde se verifica el comportamiento eléctrico y funcional de cada chip. Estas pruebas son críticas para garantizar que el chip cumpla con las especificaciones, detectando fallos de fabricación o diseño. Solo los chips que pasan estas pruebas se envían para su distribución y uso final.

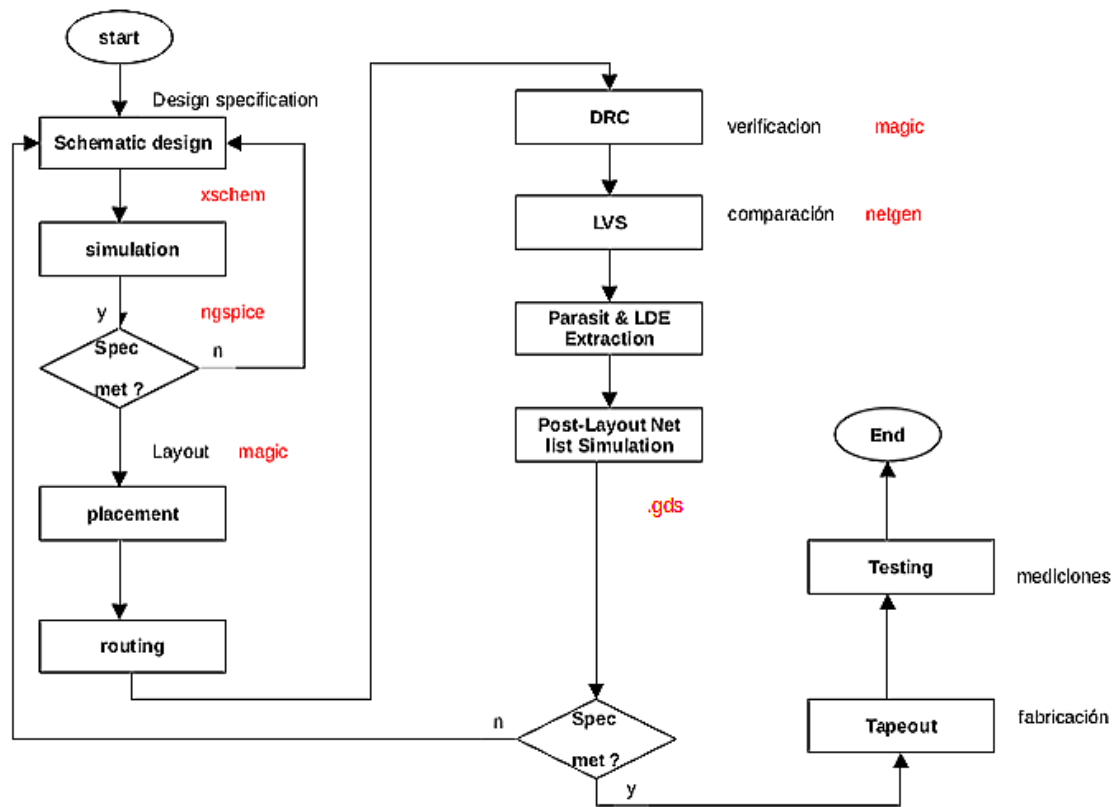


Figura 7: Modelo de Flujo de diseño analógico.

2. Consigna del TP: Especificaciones de Diseño.

El inversor CMOS a diseñar debe cumplir las siguientes especificaciones:

- Tecnología: SKY130 (130 nm CMOS).
- Voltaje de alimentación (VDD): 1.8 V.
- Características del transistor PMOS:
 - $L = 0,15$ (Longitud de canal del transistor en micrómetros)
 - $W_p = 2,1$ (Ancho del canal del transistor en micrómetros)
 - $nf = 1$ (Número de fingers)
 - $mult = 1$ (Multiplicidad)
 - model: *pfet_01v8*
- Características del transistor NMOS:
 - $L = 0,15$
 - $W_n = 1,05$
 - $nf = 1$
 - $mult = 1$
 - model: *nfet_01v8*
- Puede ajustar los valores anteriores para que la Relación efectiva W_p/W_n equilibre tiempos de subida y bajada.
- Ajustar Capacidad de carga de salida: 2 pF (considerando fan-out).
- Tiempo de transición objetivo: menor a 50 ns para subida y bajada.

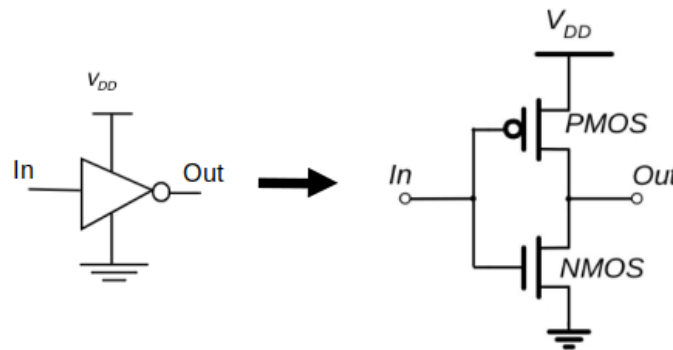


Figura 8: símbolo del inversor y circuito propuesto para el diseño con CMOS.

3. Actividad 1: Instalación de las herramientas.

Para el diseño y simulación del inversor CMOS se utilizaron herramientas opensource compatibles con el PDK SKY130. Entre ellas, se instalan:

- **Xschem** para diseñar y simular esquemáticos.
- **Ngspice**: modelos spice para simulación eléctrica en el entorno de Xschem.
- **Magic** para ruteo y layout, servirá además para verificación del diseño físico (DRC).
- **Netgen** para realizar la comparación LVS (Layout versus Schematic).

Se recomienda instalar estas herramientas en un entorno Linux y verificar su funcionamiento básico antes de comenzar el diseño.

3.1. Pasos para instalar las herramientas necesarias.

1. Primeramente actualizar repositorios:

```
sudo apt-get update
sudo apt upgrade
```

2. Será necesario contar con alguna librería gráfica como openGL:

```
sudo apt install freeglut3-dev
```

Para verificar si se ha instalado se debe hacer:

```
dpkg -L freeglut3-dev
```

3. Instalar herramientas de compilación necesarias:

```
sudo apt-get install build-essential
```

4. Instalar git:

```
sudo apt install git-all
```

5. Para verificar que la instalación fue exitosa, ejecutar:

```
git --version
```

6. Clonar repositorio:

```
git clone https://github.com/AlejandroJuarezLora/iic-osic-tool-Microse-lab.git
```

7. instalar las herramientas (Xschem,Ngspice,Magic,Netgen) con el siguiente comando:

```
./iic-osic-setup.sh
```

8. inicializar para empezar a trabajar(esto se deberá hacer primeramente siempre que necesitemos usar alguna herramienta de las instaladas anteriormente)

```
source ./iic-init.sh
```

Opcional: puede modificar directamente el archivo **.bashrc**, es decir abrir con algun editor de texto dicho archivo:

```
gedit .bashrc
```

Ir hasta el final de la linea y escribir **source ./iic-init.sh** , y guardar los cambios.

De esta manera, la próxima vez que abrimos terminal directamente ejecutamos el comando para abrir la herramienta de diseño a usar (xschem,magic,etc).

4. Actividad 2: Flujo de Diseño Analógico.

4.1. Schematic Design

Se describe a continuación el procedimiento para desarrollar el inversor CMOS. Se recomienda consultar documentación del software desde la web http://repo.hu/projects/xschem/xschem_man/xschem_man.html

1. abrir una terminal y ejecutar el comando:

```
xschem
```

2. Se abrirá la herramienta **xschem** para diseñar el esquemático, tal como se observa en la figura 9.

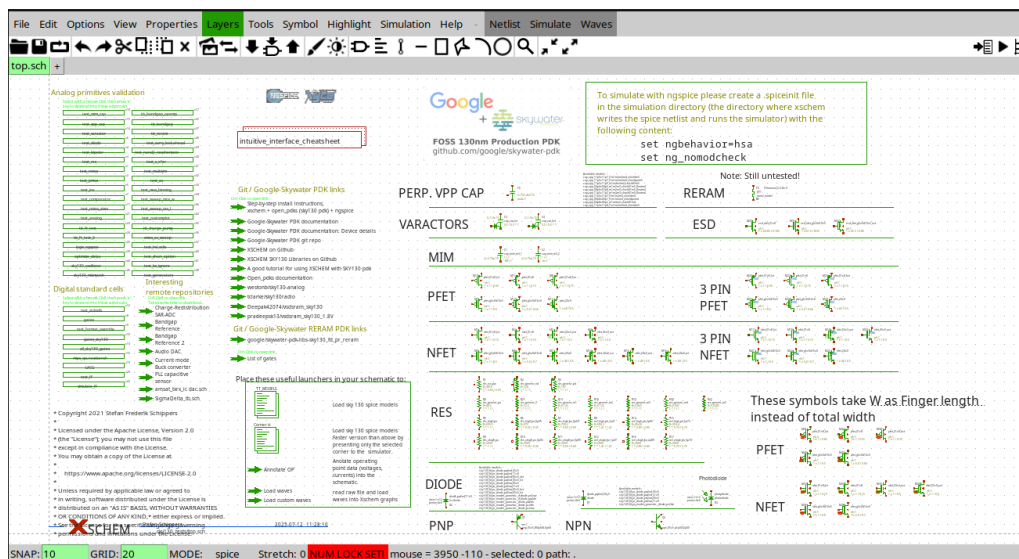


Figura 9: pantalla principal de xschem.

3. Crear un nuevo esquema. En el menú de *xschem*, crea un nuevo archivo de esquema, para ello tiene que ir a **File** → **Create a new window/tab**
4. Abre la ventana de símbolos con **shift+I** y selecciona el transistor NMOS (sky130_fd_pr_nfet_01v8) y PMOS (sky130_fd_pr_pfet_01v8).
5. Conectar el inversor: Coloca un PMOS conectado a la alimentación positiva (VDD) y un NMOS conectado a tierra (GND). Conecta las compuertas (gate) de ambos transistores a la entrada. Une los drains para formar la salida. Conecta el bulk del PMOS a VDD y el bulk del NMOS a GND.
6. Colocar pines: Agrega pines de entrada (ipin), salida (opin), VDD y GND desde la biblioteca.
7. Guardar el esquema: Guarda el trabajo realizado en el esquema, el esquemático debe quedar acorde a la figura 10.
8. Crear el símbolo: Usa la opción **Symbol** → **Make symbol from schematic** o presiona **A** para generar el símbolo asociado.
9. Confirma la creación del símbolo. Se generará automáticamente un archivo de símbolo (.sym) asociado al esquema.
10. Edita la apariencia del símbolo:
 - Usa las herramientas de dibujo para diseñar la forma del inversor (por ejemplo, un triángulo con un círculo pequeño en la salida).
 - Ajusta la posición y tamaño de los pines para que sea claro y funcional.
11. Asegúrate de que los pines de entrada, salida, VDD y GND estén presentes, visibles y correctamente nombrados para que coincidan con el esquema.
12. Guarda los cambios en el archivo del símbolo. En la figura 11 se muestra como debe quedar.
13. Utiliza el símbolo generado para instanciar el inversor en esquemas más grandes o jerárquicos con consistencia garantizada.

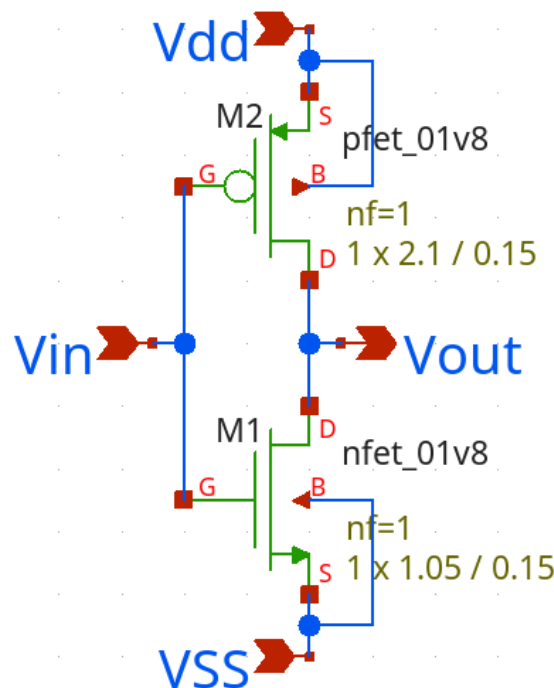


Figura 10: Circuito esquemático.

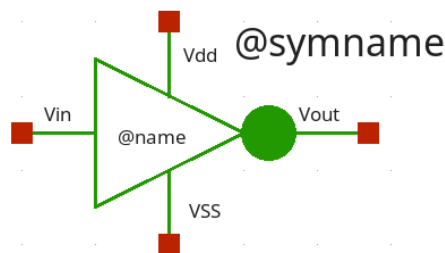


Figura 11: Símbolo del inversor creado.

4.2. Simulation

Simular el comportamiento con *Ngspice*, verificando que la transferencia de tensión de entrada a salida sea la esperada (inversión). Para ello generar un archivo *spice* desde *xscem* para simular el inversor y verificar su funcionamiento. Para mayor detalle consultar la web <https://ngspice.sourceforge.io/devel.html> o consultar manual desde <https://ngspice.sourceforge.io/docs/ngspice-html-manual/manual.xhtml>.

1. Agregar fuentes y pines: Inserta dos fuentes de voltaje (**vsources**) y los pines necesarios (**ipin** para entrada, **opin** para salida, además de **VDD** y **GND**) desde la biblioteca de símbolos.
2. Conectar el circuito: Conecta la fuente **vsources** de alimentación (**VDD**) y la fuente de señal de entrada al inversor, además de tierra, usando **lab_pins** para mantener el esquema ordenado y las señales limpias.
3. Configurar la fuente de señal de entrada: Da doble clic sobre la fuente de señal y asigna una onda de pulsos mediante el formato *ngspice*, por ejemplo: `PULSE(0 1.8 0 1n 1n 5n 10u 4)` que define los niveles bajo y alto, tiempos de subida, caída, ancho y período.
4. Configurar la fuente de alimentación: Asigna un valor fijo, por ejemplo 1.8V para la fuente de alimentación **VDD**.

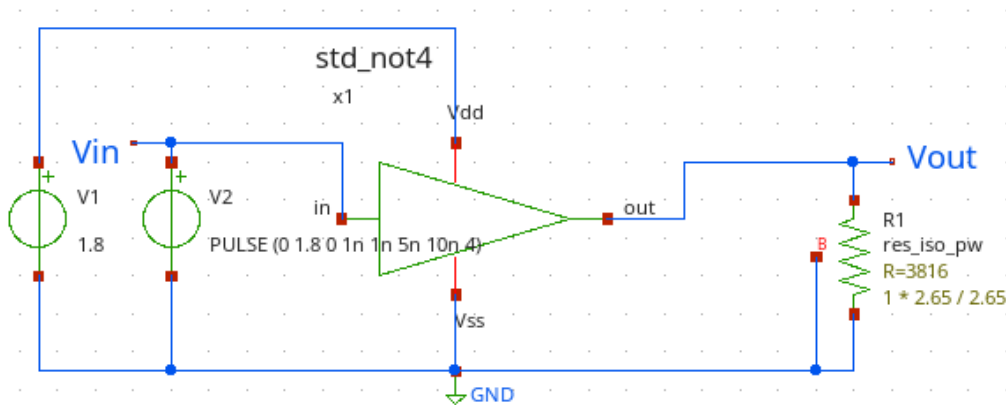


Figura 12: Montaje de circuito para simular.

- Incluir archivos de corner y modelos: Agrega el archivo de corner (ej. `corner tt`) del PDK Sky130 para que el simulador entienda correctamente los modelos y condiciones de simulación.
- Configurar tipo de simulación: Define en la configuración del simulador (ngspice) el tipo de simulación deseada (transitoria, DC, etc.) mediante el archivo netlist que genera xschem.

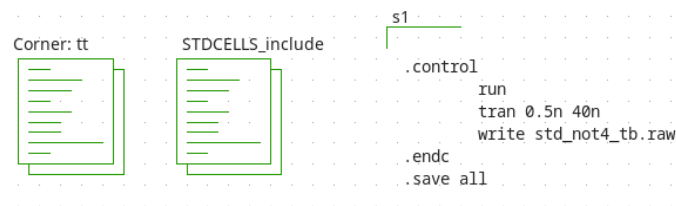


Figura 13: configuración de las simulaciones.

- Ejecutar la simulación: Corre la simulación y observa los resultados, como la respuesta en tiempo. Queda de manera opcional realizar la simulación gráfica de la curva Transferencia de Voltaje.

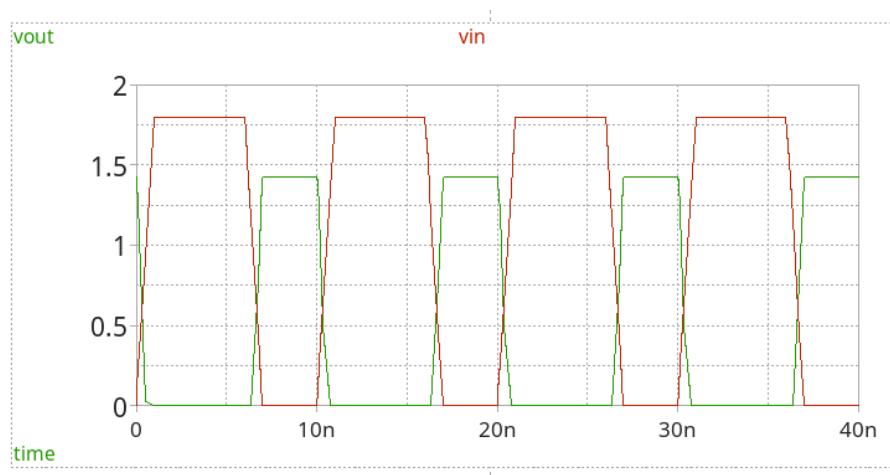


Figura 14: Plot de la simulación realizada.

- Analizar y ajustar: Con base en los resultados, ajusta parámetros del diseño, señales o fuentes para verificar diferentes condiciones y validar el comportamiento.

4.3. Layout: Placement / Routing

Realizar el diseño físico de ambos transistores siguiendo las reglas del PDK SKY130, cuidando las distancias mínimas y la correcta disposición de contactos. Consulta más información en el sitio web <http://opencircuitdesign.com/magic/>.

1. Preparar el ambiente de layout:

Abre la herramienta Magic para realizar el layout. Para iniciar, ejecuta en consola:

```
magic
```

Esto abre la interfaz de Magic tal como se muestra en la figura 15.

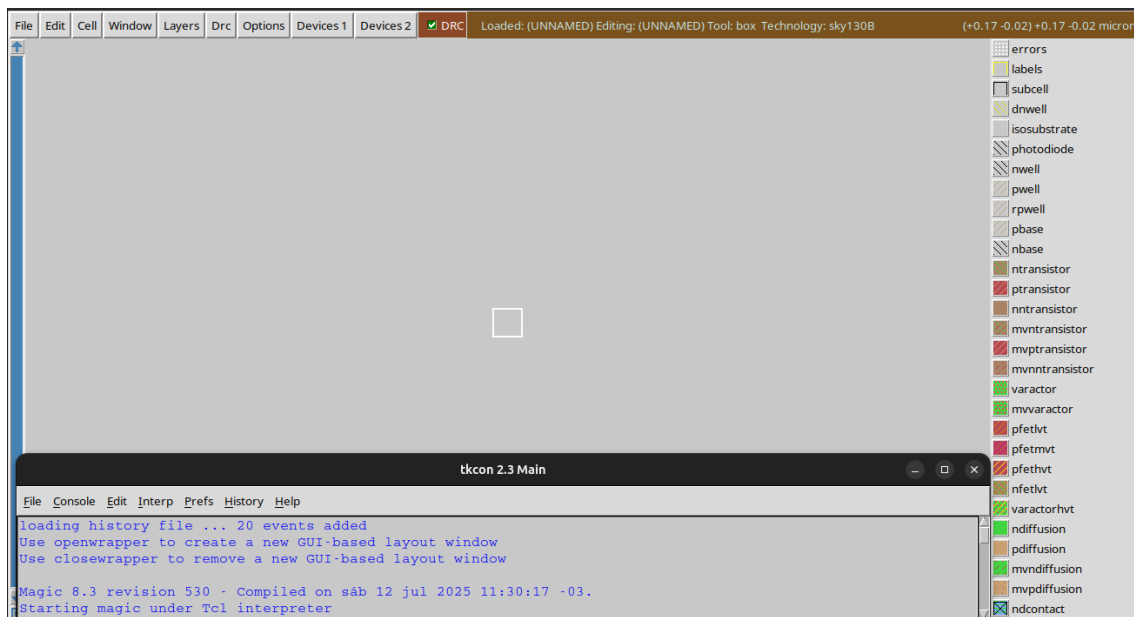


Figura 15: Pantalla principal de Magic.

Luego importa la celda diseñada con desde la barra de menú, de la ventana principal:

```
File -> Import SPICE
```

Buscar el archivo generado anteriormente y simulado, ya que el mismo contiene el netlist del circuito.

2. Placement (Colocación):

- Verifica que los transistores estén ubicados en las regiones correctas (p-well para PMOS arriba y n-well para NMOS abajo) de forma coherente con la organización del chip.
- Ajusta manualmente la posición de cada transistor para optimizar la colocación, manteniendo una altura uniforme (por ejemplo, $1.2 \mu\text{m}$) para facilitar interconexiones.
- Añade las líneas de alimentación VDD arriba y GND abajo, usando comandos como:

```
draw metal1 rectangle x0 y0 x1 y1
label metal1 "VDD"
draw metal1 rectangle x0 y0 x1 y1
label metal1 "GND"
```

- Alinea las compuertas (gates) verticalmente para simplificar el enrutamiento, modificando la posición de los transistores según convenga.

3. Routing (Enrutamiento):

- Conecta las compuertas entre sí con polisilicio utilizando:
`connect poly gate1 gate2`
 para formar las entradas comunes definidas en el circuito.
- Une los drains de PMOS y NMOS con metal o capa local para crear la salida con:
`connect metal1 drain_pmos drain_nmos`
- Traza rutas de metal para las líneas de alimentación VDD y GND, conectando las fuentes de los transistores:
`draw metal1 path x0 y0 x1 y1 x2 y2`

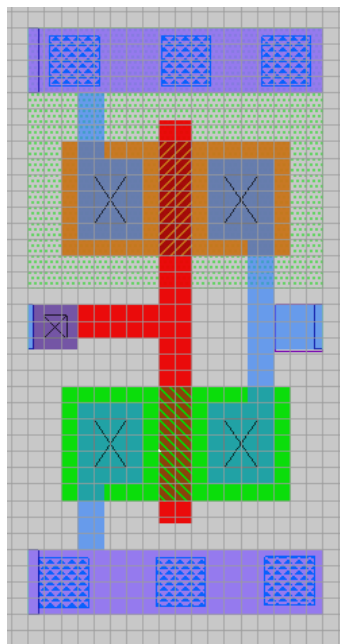


Figura 16: layout del inversor.

Nota: la imagen de la figura 16 es una representación de alternativa de routing, este paso siempre queda a elección del diseñador.

4.4. DRC

Verificación de reglas de diseño (DRC).

- Ejecuta la verificación DRC dentro de Magic con:

```
drc check
```

Esto detecta violaciones como capas sobrepuestas, distancias insuficientes o contactos incorrectos. Para visualizar errores usa:

```
drc show
```

Corrige todos los errores hasta obtener un reporte limpio para evitar problemas en fabricación, puede ver mas detalles en <https://skywater-pdk.readthedocs.io/en/main/rules.html>.

4.5. LVS

Verificación de correspondencia (LVS) entre layout y esquemático.

1. Generar netlists:

- En Magic extrae el netlist para LVS con:

```
extract all
ext2spice -lvs
```

Generará un archivo `layout.spice` con la información del layout.

- Exporta el netlist del esquemático desde `xschem` con:

```
xschem --export ngspice schematic.sch -o schematic.spice
```

Esto crea el archivo `schematic.spice`.

2. Preparar archivos para Netgen:

- Asegúrate que ambos netlists (`layout.spice` y `schematic.spice`) están en formato SPICE.
- Descarga o verifica el archivo de configuración del proceso, generalmente llamado `sky130A_setup.tcl`, que contiene las reglas para la comparación con Netgen.

3. Ejecutar Netgen: Abrir terminal y ejecutar:

```
netgen
lvs layout.spice schematic.spice sky130A_setup.tcl
```

Esto realiza la comparación automática entre el netlist del layout y del esquema.

4. Interpretar resultados:

- En caso de éxito, verás un mensaje que confirma la coincidencia entre layout y esquemático.
- Si hay discrepancias, Netgen genera un reporte indicando errores, por ejemplo:

```
Device mismatch on transistor M1
Connection missing at node out
```

5. Iterar correcciones:

- Corrige los errores reportados en el layout o esquemático.
- Repite la extracción y la ejecución de Netgen hasta que el LVS sea exitoso.

Para mayor detalles visitar el sitio <http://opencircuitdesign.com/netgen/index.html>

4.6. Post-layout

Realizar simulaciones post-layout para validar el comportamiento eléctrico considerando efectos y parasitismos del layout.

1. **Extracción de parásitos:** La extracción de elementos parasitarios es fundamental para capturar las resistencias y capacitancias introducidas por las interconexiones y dispositivos en el layout final. Para ello, utiliza herramientas especializadas como Magic. En Magic, ejecuta primero el comando:

```
extract all
```

para realizar la extracción completa de geometrías y parasitismos. Luego, genera un netlist que incluya estos elementos con:

```
ext2spice
```

o si quieres un netlist específico para LVS:

```
ext2spice -lvs
```

Este proceso crea un netlist detallado que modela de manera precisa los efectos del layout, incluyendo resistencias parasitarias de metal y capacitancias entre capas.

2. **Generación del netlist post-layout:** Combina el netlist extraído con los modelos del proceso del PDK Sky130 que contienen parámetros físicos y eléctricos de los dispositivos (NMOS, PMOS, resistencias, capacitores, etc.). Esto usualmente se logra incluyendo los modelos del PDK en el archivo de simulación SPICE o agregándolos como archivos aparte al simulador. De esta manera, el netlist post-layout no solo incluye la topología y componentes, sino también sus características eléctricas reales con parasitismos.
3. **Configuración y ejecución de la simulación:** Utiliza un simulador ngspice para ejecutar simulaciones transitorias, DC o AC en el netlist post-layout. Por ejemplo, desde consola puedes ejecutar:

```
ngspice postlayout.spice
```

donde el archivo `postlayout.spice` contiene el netlist combinado con modelos y parasitismos. Realiza análisis de:

- Retardos de propagación (timing).
- Integridad de señal (voltajes, ruido).
- Consumo de potencia.

Esto permite evaluar cómo afectan los efectos del layout al rendimiento del circuito.

4. **Análisis de resultados y ajustes:** Compara los resultados de simulación post-layout con los obtenidos en la simulación pre-layout (solo esquemáticos). Si detectas desviaciones significativas, ajusta el diseño o modifica el layout para mejorar la integridad y funcionamiento:
 - Optimiza rutas para reducir resistencias parasitarias.
 - Reubica componentes para mejorar conexión y reducir capacitancias no deseadas.
 - Repite la extracción y simulación hasta alcanzar los objetivos eléctricos deseados.

4.7. Generación del archivo .gds para fabricación

Para generar el archivo `.gds` (es el que se envía a la foundry) una vez que se han validado los pasos anteriores, se tienen que seguir los siguientes pasos:

1. Abre la herramienta *Magic* y carga el layout final que deseas exportar.
2. Asegúrate de que el diseño esté completo y sin errores. Es recomendable ejecutar una última verificación con:

```
check
```

para detectar errores visibles que puedan afectar la fabricación.

3. En la consola de comandos ejecuta:

```
writeall nombre_diseño.gds
```

Donde `nombre_diseño.gds` es el nombre que quieres darle al archivo de salida.

El comando `writeall` exporta todo el contenido del layout, incluyendo todas las capas designadas en el proceso, y las organiza en el formato estándar (.gds), que contiene toda la información geométrica necesaria para la fabricación del chip.

4. Verifica que el archivo `nombre_diseño.gds` se haya generado correctamente en el directorio de trabajo actual.
5. Finalmente, ese archivo es el que se envía a la foundry para que inicie el proceso de fabricación del chip.

5. Conclusiones.

El propósito principal de este Trabajo Práctico ha sido familiarizar al estudiante con un flujo de trabajo básico en el diseño analógico de circuitos integrados, brindando las herramientas (opensource) y conocimientos fundamentales empleados durante el proceso de desarrollo.

Se busca que el alumno pueda apreciar la complejidad involucrada y el rigor necesario en cada fase del flujo de trabajo. Más allá de haber sido un simple ejercicio académico, esta experiencia abre las puertas a un campo tecnológico fascinante y en constante evolución, donde la miniaturización, la precisión y la eficiencia energética son prioritarias.

Por lo tanto, servirá como una introducción que invita a seguir profundizando en temas avanzados de diseño integrado, seguridad en manufactura, optimización de parámetros eléctricos y nuevas tecnologías. La práctica, el estudio continuo y la experimentación serán fundamentales para desarrollar competencias sólidas y profesionales capaces de enfrentar los retos del diseño de circuitos integrados analógicos modernos.

Finalmente, se recomienda aprovechar los recursos disponibles, como las herramientas libres (Magic, Netgen, ngspice, etc.) y la documentación del PDK abierto Sky130, todo esto le permitirán seguir explorando y aplicando estos conocimientos de forma accesible y práctica.

Se espera que esta experiencia motive a los estudiantes a avanzar en el área, inspirando creatividad, rigor técnico y pasión por el diseño de circuitos integrados en un entorno real de ingeniería.

Para concluir y a partir de sus observaciones durante el proceso de diseño realizado, deberá responder a modo de conclusión las siguientes preguntas:

- ¿Cuáles fueron los principales desafíos encontrados durante el diseño y las simulaciones del circuito analógico CMOS?
- ¿Cómo influyeron las reglas de diseño físico y las características del proceso CMOS en las decisiones del diseño?
- ¿Qué parámetros eléctricos y físicos del circuito fueron los más críticos para cumplir con las especificaciones iniciales?
- ¿Cómo se logró validar el funcionamiento del circuito a través de simulaciones antes del diseño físico?
- ¿Qué aprendizajes se obtuvieron sobre la importancia de la integración entre diseño esquemático, simulación y layout?
- ¿Cómo puede este diseño aplicarse a situaciones reales o a otros proyectos de circuitos analógicos?
- ¿Qué mejoras o ajustes considerarías para futuros diseños similares basados en la experiencia de este TP?

6. Bibliografía / Referencias.

- Rabaey, J. M. (1995). Digital integrated circuits: A design perspective (First edition). Prentice Hall.
- Razavi, B. (2017). Design of analog CMOS integrated circuits (2nd ed.). McGraw-Hill Education.
- <http://repo.hu/projects/xschem/>
- <https://ngspice.sourceforge.io/devel.html>
- <https://skywater-pdk.readthedocs.io/en/main/>
- <http://opencircuitdesign.com/>

7. Anexos

7.1. Resumen de comandos básicos de Magic

Comando	Descripción
<code>magic -d XR</code>	Inicia [translate:Magic] en modo gráfico con soporte para reglas de diseño.
<code>load <nombre_celda></code>	Carga una celda o diseño existente para editar.
<code>save</code>	Guarda los cambios realizados en el diseño abierto.
<code>extract all</code>	Extrae la información del layout necesaria para generar netlists.
<code>ext2spice -lvs</code>	Genera el netlist para la verificación LVS a partir de la extracción.
<code>drc check</code>	Ejecuta la verificación de reglas de diseño (DRC) para detectar errores.
<code>drc show</code>	Muestra en el layout los errores detectados en la verificación DRC.
<code>check</code>	Realiza una comprobación rápida de diseño para detectar posibles errores.
<code>create transistor pmos <X><Y><ancho><alto></code>	Crea un transistor PMOS en las coordenadas especificadas.
<code>create transistor nmos <X><Y><ancho><alto></code>	Crea un transistor NMOS en las coordenadas especificadas.
<code>draw metal1 path <x0><y0><x1><y1>...</code>	Dibuja una ruta de metal para conexiones eléctricas.
<code>connect <capa><terminal1><terminal2></code>	Conecta dos terminales eléctricamente usando la capa especificada.
<code>quit</code>	Cierra la aplicación [translate:Magic].
<code>undo / Ctrl+Z</code>	Deshace la última acción realizada.
<code>redo / Ctrl+Y</code>	Rehace la acción deshecha.
<code>save / Ctrl+S</code>	Atajo para guardar cambios rápidamente.
<code>m</code>	Selecciona la herramienta mover para desplazar objetos.
<code>c</code>	Copiar objetos seleccionados.
<code>x</code>	Cortar objetos seleccionados.
<code>v</code>	Pegar objetos copiados o cortados.
<code>r</code>	Rotar objetos seleccionados.
<code>snap</code>	Activa o desactiva el ajuste a la cuadrícula para un posicionamiento preciso.

Cuadro 1: Algunos comandos en Magic. Más información en <http://opencircuitdesign.com/magic/>

7.2. Resumen atajos usados en Xschem

Comando / Atajo	Descripción
Ctrl + N	Crear un nuevo esquemático.
Ctrl + O	Abrir un esquemático existente.
Ctrl + S	Guardar el esquemático actual.
Ctrl + Z	Deshacer la última acción.
Ctrl + Y	Rehacer la última acción deshecha.
A	Añadir un componente (abre el navegador para seleccionar componentes).
G	Colocar componente seleccionado en el canvas.
R	Rotar componente seleccionado 90° en sentido horario.
M	Mover el componente seleccionado.
Delete	Eliminar el elemento seleccionado.
W	Añadir una conexión (wire) entre nodos.
F	Buscar componentes o netnames.
Ctrl + E	Ejecutar simulación con el simulador configurado (generalmente ngspice).
Ctrl + P	Ajustar propiedades del componente seleccionado.
Ctrl + G	Agrupar componentes seleccionados.
Ctrl + L	Agregar texto libre (labels o notas).
Ctrl + Q	Salir del programa.

Cuadro 2: detalles en https://xschem.sourceforge.io/stefan/xschem_man/xschem_man.html.

7.3. Comandos para simulaciones en ngspice

Comando / Atajo	Descripción
run	Ejecuta la simulación definida en el archivo SPICE cargado.
load <archivo.spice>	Carga un archivo SPICE para simularlo.
source <archivo>	Ejecuta comandos desde un archivo externo en la sesión actual.
plot <variable>	Grafica una variable (voltaje, corriente) especificada.
print <variable>	Muestra en texto el valor de una variable en la simulación.
alter	Permite modificar el circuito sin salir de ngspice para realizar simulaciones con cambios menores.
display	Muestra información sobre el circuito actual, como nodos y elementos.
help	Muestra ayuda con comandos disponibles en ngspice.
quit o exit	Salir de la sesión de ngspice.
set	Configura opciones del simulador, como precisión o límites de tiempo.
ic	Define condiciones iniciales para nodos en simulaciones transitorias.
tran <tstep><tstop>	Realiza un análisis transitorio desde 0 hasta tstop con paso tstep.
dc <source><start><stop><step>	Realiza un análisis DC barriendo el valor de una fuente.
ac <type><points><fstart><fstop>	Ejecuta análisis en frecuencia (AC sweep) entre fstart y fstop.
save	Guarda el estado actual o variables para análisis posterior.

Cuadro 3: Detalles en <https://ngspice.sourceforge.io/docs/ngspice-manual.pdf>.